NORWEGIAN UNIVERSITY OF LIFE SCIENCES
(NMBU)

REPORT: DYNAMICAL LOW-RANK TRAINING

DYNAMO: A SOFTWARE SOLUTION FOR OPTIMIZING DYNAMIC LOW RANK NEURAL NETWORKS.

AUTHORS

LEO QUENTIN TH. BÆKHOLT
SIMEN ROKO KROGSTIE
PAVEL GULIN

NORWAY, MARCH 2025

This page intentionally left blank.

Contents

This page intentionally left blank.

1 Introduction

Recent developments in artificial intelligence, particularly in deep neural networks, have led to significant technological advancements. It has, however, come at a significant cost. Along with increasing complexity of the neural network architectures, comes increased computational expense, presenting a significant challenge in terms of energy efficiency and environmental impact.

Addressing this critical issue, this report details the implementation of a software package using PyTorch, aimed at optimizing the efficiency of deep neural networks while mitigating their environmental impact. Our approach harnesses the power of low-rank neural networks, a method that significantly reduces memory and computational overhead by employing low-rank factorizations of weight matrices. This approach effectively reduces computational requirements and simultaneously supports energy conservation efforts in the development and deployment of deep neural networks.

2

Background

A neural network is constructed using a series of layers connected through neurons. The first layer of a neural network is called the input layer, and it receives the input data. In the input layer, each neuron represents one feature in the input data. Secondly the neural network has as an output layer, in which produces the output of the network. In between the input layer and output layer we have so-called hidden layers. A neural network can consist of one or more such hidden layers. The neurons in these layers take the weighted sum of the outputs from the layer before and process it through a non-linear activation function. A commonly used activation function is ReLU. They then pass the output to the next layer. The general formula for a neural network can be expressed as:

$$\mathbf{h}^{(l)} = \sigma(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)})$$

Where σ is the nonlinear activation function, W is the weight matrix, and b is the bias term.

To ensure the images are being labelled correct we must train the model. With training of a neural network, we want to measure the predictions of correct labels and images by defining a loss function. The loss function measures how close the model matches the labels and images in the dataset. The goal is to minimize this loss function. In classifying problems cross entropy is commonly used as the loss function.

To minimize the loss function we must determine the weights W and bias b that minimizes the loss function. We can do this by finding the partial derivatives of the loss function with respect to all of the weights, by backpropagating errors, and then applying a gradient based optimization algorithm to find an approximation of the loss function's minima. The backward propagation begins at the output layer of the network and moves through the network layer by layer backwards. One of the simplest and most used optimizers is the gradient descent optimizer. To update the weights, iteratively we subtract a fraction of the gradients, which is determined by the learning rate (λ), from the weights.

$$W := W - \lambda \nabla_W \text{Loss}$$

As explained in the introduction, with the increasing complexity of neural networks, an increased computational expense is also arising. However, by reducing the usage of memory in neural networks, the computational expenses can also be reduced, subsequently reducing power usage and hardware usage. This contributes to lowering the environmental impact artificial intelligence and deep learning have. To reduce the memory usage in a neural network, you can train low-rank factorizations of the weight matrices. Such low-rank neural network is constructed very similar to a standard neural network. However, the difference is that we decompose the weights to a product of smaller matrices. The formuala for a low-rank neural network can be expressed as:

$$W \approx USV^t$$

The training of a low-rank neural network vanilla style is not very different from training a standard neural network. Now we train each of the decomposed matrices U, S and V in each layer. Hence, we must compute the gradient for each of these matrices to get the loss function. However, the vanilla training has a downside. That is, for the loss function to reach its minimum, we must use a very small learning rate λ. This means the number of epochs must be large, which leads to high computational costs. To achieve training of the low rank neural network without having high computational costs, we can train the network dynamically. This type of training utilizes more complex mathematics which enables the network to use a higher learning rate during training. Thus we lower the number of epochs and computational costs.

3

Quick Start Guide: Your First Network

This neural network framework is designed to offer a streamlined, configurable approach to building and training neural networks. The framework also introduces innovative components such as dynamic low rank layers, and an optimizer which can assign different optimizers to different layers. By leveraging a TOML configuration file, users can define complex neural network architectures and training parameters in a simple, readable format. Compatibility with the novel dynamic low rank layer is ensured in every part of the software.

3.1 Installing the Framework

To ensure a smooth installation process, it's recommended to start in a fresh virtual environment. This approach isolates the framework and its dependencies from your global Python environment, preventing potential conflicts. Follow these steps to install the framework:

1. **Create a Virtual Environment:** Create and activate a fresh virtual environment. Both conda and venv work.
2. **Install Poetry:** Poetry is a tool for dependency management and packaging in Python. Install it using `pip install poetry`.
3. **Install Dependencies:** Run `poetry install` in the root directory of the framework. This command fetches and sets up all necessary packages, ensuring that you have a ready-to-use environment for your neural network projects.

Alternatively: If you are struggling with making poetry run (it sometimes has issues with finding the correct interpreter and whatnot), you can replace points **2** and **3** with running `pip install -r requirements.txt` at the root of the project.

3.2 Configuring the Network

After installation, the next step is to configure and run your neural network. Configuration of the network is done through a TOML file, which provides a clear and editable

format for defining your network's structure and behavior. Below is an example of a typical configuration:

Listing 3.1: *Sample TOML Configuration*

```
[settings]
batchSize = 64
numEpochs = 10
architecture = 'FFN'

# ------------ Layers ------------

[[layer]]
type = 'lowRank'
dims = [784, 64]
activation = 'relu'
rank = 20

[[layer]]
type = 'dense'
dims = [64, 10]
activation = 'linear'

# ------------ Optimizers ------------

[optimizer.default]
type = 'SimpleSGD'
parameters = { lr = 0.005 }

[optimizer.lowRank]
type = 'DynamicLowRankOptimizer'
parameters = { lr = 0.1 }
```

You can modify this file to change the network architecture, layer types, dimensions, activation functions, and optimizer settings as per your project needs. The output of any layer will be the input of the next. Any layer which doesn't have an otherwise specified optimizer will be assigned to `optimizer.default`.

3.3 Executing the Network

From here, you have a few different choices. Using command line arguments, you can either train with a base config, your own custom config, all the config files in a directory, or run the software's GUI. Refer to the file `README.md` for specifics.

Listing 3.2: *Sample Code for instanciating network*

```
# Replace 'PATH_TO_YOUR_CONFIG' with the actual path to your configuration
    file
model = FeedForward.create_from_config("PATH_TO_YOUR_CONFIG")

# Initialize the trainer with your model
trainer = Trainer.create_from_model(model)

# Replace 'your_train_dataloader' and 'your_test_dataloader' with your
    actual data loaders
trainer.train(your_train_dataloader, your_test_dataloader)
```

4

Software Architecture and Design Choices

4.1 Modularity

The design of DynamO is grounded in modularity, enabling it to adapt and integrate with a wide range of neural network architectures. This flexibility is key to its utility in various applications, far beyond trivial examples like MNIST training with dynamic low-rank layers.

4.1.1 Beyond Basic Configurations

While DynamO allows for straightforward network configuration and training through a single config file, its true potential likely lies in its application to more complex scenarios. Because of the software's modularity, it would be possible to integrate it with more complex architectures. One example, that could possibly work very well, is using a pretrained backbone together with the low rank layers. By taking a robust, pretrained backbone, freezing its weights, and appending a few dynamic low-rank layers, DynamO could hopefully transform the network into a powerful yet efficient image classifier. This approach could drastically reduce the number of parameters needing in training.

4.1.2 Compatibility and Integration

One of the standout features of DynamO is the broad compatibility of its dynamic low-rank layers and MetaOptimizer. These components can be seamlessly integrated into almost any architecture, provided that all layers are set as attributes in the model. This universality means that DynamO can be a valuable tool across a wide array of neural network designs, from custom architectures to industry-standard models. On the next page is a showcase of how simple it is to implement dynamic low rank layers into any existing neural network using DynamO:

Listing 4.1: *Dense Layers*

```
class SimpleCNN(nn.Module):
    def __init__(self):
        super(NetDense, self).
            __init__()
        self.c1 = nn.Conv2d(1, 32,
            5)
        self.c2 = nn.Conv2d(32, 64,
            5)
        self.fc1 = nn.Linear(1024,
            128)
        self.fc2 = nn.Linear(128,
            10)
        self.pool = nn.MaxPool2d(2,
            2)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.pool(self.relu(self
            .c1(x)))
        x = self.pool(self.relu(self
            .c2(x)))
        # Flattening and Dense
            layers
        x = torch.flatten(x)
        x = self.relu(self.fc1(x))
        x = self.fc2(x)

net = SimpleCNN()
Optimizer = torch.optim.Adam(
    SimpleCNN.parameters(), lr=3e-4)

# Training loop: ...
```

Listing 4.2: *Dynamic Low Rank Layers*

```
class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).
            __init__()
        self.c1 = nn.Conv2d(1, 32,
            5)
        self.c2 = nn.Conv2d(32, 64,
            5)
        self.dlr1 =
            DynamicLowRankLayer(1024,
             128, rank=20)
        self.fc1 = nn.Linear(128,
            10)
        self.pool = nn.MaxPool2d(2,
            2)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.pool(self.relu(self
            .c1(x)))
        x = self.pool(self.relu(self
            .c2(x)))
        x = torch.flatten(x)
        x = self.relu(self.dlr1(x))
        x = self.fc1(x)
        return x

net = SimpleCNN()

optimizer_config = {
    "default": (torch.optim.Adam, {'
        lr': 3e-4}),
    DynamicLowRankLayer: (
        DynamicLowRankOptimizer, {'lr
        ': 3e-2}),
}

optimizer = MetaOptimizer(net,
    optimizer_config=optimizer_config
    )

# Training loop: ...
```

4.2 Configuration-Driven Architecture

4.2.1 TOML-Based Network Definition

The framework utilizes a TOML file to define network architectures. This configuration-centric approach allows for easy definition of network parameters, layers, hyperparame-

ters and training parameters. An example TOML file is provided below:

4.2.2 ConfigParser Module

The `ConfigParser` module plays a pivotal role in translating the TOML configuration file into a functional neural network architecture. This module streamlines the process of setting up a neural network, making it accessible and efficient for users to define complex architectures without delving into the intricacies of manual setup.
Functionalities and Workflow:

- **Parsing the Configuration:** The `ConfigParser` initializes with the path to the TOML file. It reads and parses the file, extracting key information such as batch size, number of epochs, and the overall architecture of the neural network.
- **Layer Mapping:** A significant feature of `ConfigParser` is its ability to map string representations of layers and optimizers to their respective classes in the codebase. This includes standard layers like `DenseLayer`, as well as specialized ones like `DynamicLowRankLayer`. This mapping is extendable, allowing for future expansions and custom layer types. If one wanted to, say, add support for convolutional layers, all one would need to do would be to either add `"conv2d": nn.Conv2d` to the dictionary named `self.layer_class_mapping`, or add it to the dictionary by calling the method: `add_layer_mapping("conv2d", nn.Conv2d)` on the parser instance. Note: if using the constructor static method on the `FeedForward` network, the network will create a fresh instance of the `ConfigParser`, so it is necessary to add new mappings directly into the dictionaries found in `__init__`
- **Layer Instantiation:** Based on the parsed configuration, `ConfigParser` dynamically creates each layer of the neural network. It handles various aspects such as layer types, dimensions, and activation functions, thus constructing the model as per user specifications. It stores all of the layer class instances in a list named `self.layers`.
- **Optimizer Configuration:** The `ConfigParser` module also takes charge of setting up optimizer configurations. It maps specific optimizers to their respective layers, a critical process especially for specialized layers like `DynamicLowRankLayer` that need a specific optimizer. This mapping is stored in `self.optimizer_config`, and will later be used as input to the `MetaOptimizer`

4.3 Optimizer Configuration and Management

4.3.1 MetaOptimizer Functionality

The `MetaOptimizer` plays a critical role in the framework by managing the optimization process across different layer types. This component's design aligns with the framework's emphasis on flexibility and efficiency, particularly when dealing with a variety of layers that require distinct optimization strategies.

- **Layer-Specific Optimization**: The `MetaOptimizer` is designed to assign specific optimizers to different layer types. This is particularly beneficial for layers like `DynamicLowRankLayer`, which necessitate unique optimization techniques. By using a configuration dictionary, it maps layer types to their corresponding optimizer classes and parameters. This configuration dictionary can either be retrieved from the `ConfigParser`, defined manually, or not defined at all (in which case, a default setting will be used). This setup allows for easy customization and extension of the optimization strategy for different layer types.
- **Alternating Training Capability**: One of the features less seen elsewhere of the `MetaOptimizer` is its support for alternating training modes. This functionality is especially advantageous for optimizing layers like `DynamicLowRankLayer`, where one needs to alternate between optimizing different sets of parameters. In this case, when the network has dynamic low rank layers, one needs to alternate between optimizing all parameters in the entire network, and only optimizing the S parameter in the dynamic low rank layers.
- **Optimization Process**: The initialization of `MetaOptimizer` involves passing the neural network model and the optimizer configuration. During the training phase, the `MetaOptimizer` iterates over each layer of the neural network, applying the appropriate optimizer based on the layer type. For layers that do not have a specified optimizer in the configuration, a default optimizer is applied, ensuring that every layer is optimized.

4.4 Graphical User Interface

4.4.1 GUI Functionality

As one of our objectives was to create a user-friendly program, we decided to develop a graphical user interface (`GUI`) for easier interaction with our software. Our `GUI` provides all the necessary parameters and more. It integrates our code to form a practical, real-life application suitable for industry use.

- **Import and Train your Model**: You can utilize our `GUI` to load and train a neural network using a config file. Additionally, our `GUI` offers the capability to load and train multiple networks simultaneously, using different config files located in the same folder. The `GUI` displays training results in real-time and provides a comprehensive summary upon completion. In scenarios involving multiple config files, our GUI outputs a list of tuples, each containing the training accuracy and the time taken for each training session.
- **Load and Save best performing Models**: Our `GUI` is equipped with the capability to load pretrained weights and biases. Additionally, it allows users to save the best-performing model to a specified folder.
- **Predict Numbers**: We have extended our program with the ability to predict numbers. The `GUI` offers two options: you can either draw a number yourself or

input an index from 0 to 9999, corresponding to a specific tensor in the MNIST dataset. The program then processes your input and outputs its prediction. For numbers derived from the MNIST dataset, it also displays the label of the image, allowing for easy comparison.

4.5 Good OOP Practices in DynamO

4.5.1 Encapsulation

We have encapsulated all functionality within well-defined classes, ensuring that each class has a distinct responsibility. For instance, the `ConfigParser` class is solely responsible for parsing the TOML configuration files, while the `MetaOptimizer` class handles the optimization logic. This encapsulation promotes modularity and makes the codebase more manageable and less prone to errors.

4.5.2 Abstraction

By abstracting complex operations, DynamO offers a simplified interface for users. Complex operations, such as setting up dynamic low-rank layers or configuring optimizers, are hidden behind straightforward method calls. This abstraction not only enhances user experience but also makes the internal workings of the system more flexible and easier to modify later.

4.6 Testing

There is a parallel directory for testing in `tests`. There are in total 44 tests. All of them can be run by running `pytest` at the root of the project.

5

Agile development

5.1 Approach and organization

We approached the project by reading and understanding the letter we received from NMBU AI, in depth. This gave us an understanding of how we could develop a software that provides the features that was specified, simultaneously as it is user friendly and extendable. Then we made an issue board in our git repository, consisting of everything necessary to develop such software. We worked accordingly to the Scrum project management framework with sprints throughout the project. We would assign each other to issues from the board before a sprint, and work with these issues for a couple of days. Below you can see what the issue board looked like during different stages of developing the software.

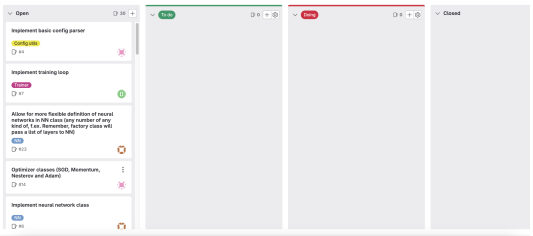

Figure 5.1: *Issue board before the first sprint.*

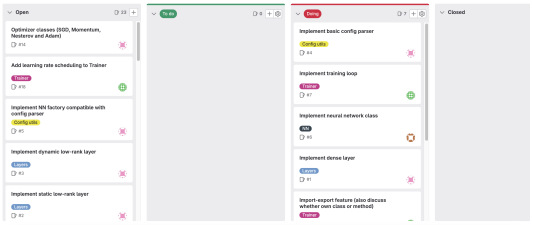

Figure 5.2: *Issue board during first sprint.*

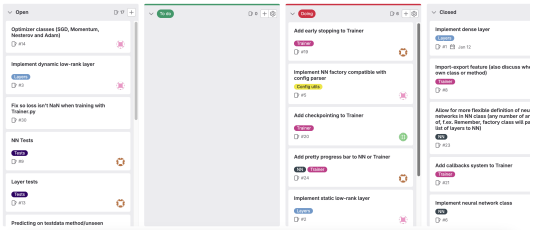

Figure 5.3: *Issue board during second sprint*

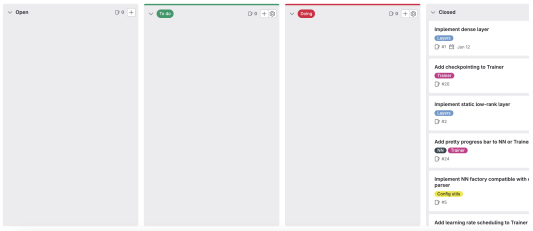

Figure 5.4: *Issue board after everything had been implemented*

6

Results

The introduction of dynamic low-rank layers into our neural network architecture, specifically designed for the MNIST dataset, has led to promising results. Our experiments show that networks with dynamic low rank layers can achieve similar performance to that of networks consisting exclusively of dense layers, with a fraction of the parameters. It is worth noting, however, that we suspect these dynamic low rank layers may have a much greater potential in larger network architectures than used here.

6.1 Getting to the numbers

To ensure a fair comparison between dense layers and dynamic low-rank layers, we opted for a straightforward and logical architecture. Our architecture consisted of a feed-forward Artificial Neural Network (ANN) with an input layer of 784 nodes (dimension of the flattened MNIST data). This was followed by two hidden layers, the first with 128 nodes and the second with 64 nodes, and finally, an output layer comprising 10 nodes. The output from the "normal", dense network was sent through a softmax function. Interestingly, while the output of the standard dense network was passed through a softmax function for class probability calculation, the dynamic low-rank network exhibited training issues with softmax normalization and was thus trained without it. Both the standard dense layer model and the dynamic low-rank layer model were trained for 30 epochs. For evaluation, we selected the validation accuracy from the epoch that yielded the best performance. Based on empirical observations, a rank of 20 was determined to be generally effective for the hidden layers in the low-rank network, and hence it was employed in our experiments.

Dense Network Model

- Total Parametes: 109386
- Input layer: 784 nodes
- First hidden layer: 128 nodes
- Second hidden layer: 64 nodes
- Output layer: 10 nodes
- Output processing: Softmax function
- Optimizer: Adam
- Learning Rate: 0.005
- Best accuracy: 94.89%

Dynamic Low-Rank Network Model

- Total Parameters: 23722
- Input layer: 784 nodes
- First hidden layer: 128 nodes (Rank 20)
- Second hidden layer: 64 nodes (Rank 20)
- Output layer: 10 nodes
- Output processing: None
- Optimizer: Dynamic Low Rank Optimizer
- Learning Rate: 0.05
- Best accuracy: 93.16%

(Note: The detailed code and figures for these results can be found in the Jupyter notebook at `report_figures_code/basicdense_vs_basic_lowrank.ipynb`.)

This goes to show that a roughly 78% reduction in parameters leads to less than a 1.5 percentage points reduction in performance. This finding highlights the efficiency of dynamic low-rank layers in maintaining a high level of accuracy while substantially decreasing the model's number of parameters.

6.2 Learning Rate and Stability

During our experimentation, we encountered an intriguing issue related to the learning rate and the stability of training in dynamic low-rank networks. We observed that when the rank of the layers was set relatively high (50+ for layers with sub 100 nodes), there was a tendency for the loss to become NaN within just a few training iterations, before even a single epoch could be completed. We found that reducing the learning rate somewhat stabilized this issue. While a lower learning rate did help in mitigating this issue, it remains a critical factor to consider in the design and training of networks with dynamic low-rank layers. This sensitivity to the learning rate underscores the importance of careful hyper-parameter tuning in achieving optimal performance and stability in such models. We also found that if the matricies U and V^t weren't initialized as orthogonal, the loss would also become NaN or extremely large.
In addition, our empirical findings suggest that it is safer to have larger layers (more hidden units) with a rank between 15-30, rather than a smaller layer (fewer hidden units) with a higher rank. This, however, is dependent on a bunch of different factors, such as the training data, learning rate and so on.

6.3 Potential in Larger Networks

The use of dynamic low-rank layers in relatively small networks, such as the ones used here, suggests their potential in larger, more complex architectures. The principle behind low-rank networks implies greater efficiency in models where there is a possibility of more redundant information. Therefore, applying dynamic low-rank layers in larger networks, where redundancy is likely to be more prevalent, could lead to even more significant reductions in parameters without a substantial compromise in performance. Future research could explore the application of these layers in more extensive and complex neural network architectures, hopefully further improving efficiency and performance.

6.4 Conclusions

In summary, our explorations into the use of dynamic low-rank layers for MNIST dataset classification have given promising results. These layers have proven effective in maintaining high performance levels while drastically reducing the number of parameters. Our findings support the viability of dynamic low-rank layers. Given the continual growth in size and complexity of neural networks, methods like dynamic low-rank factorization will likely become essential for enhancing computational efficiency and environmental sustainability of these models.